EnergyMonitor: monitoring home electricity consumption with an IoT device, a microservices architecture, and alerts based on real measurement

Juan Esteban Cortés Parra and Miguel Ángel García Artunduaga

Servicio Nacional de Aprendizaje (SENA)

Technologist in Software Analysis and Development (ADSO), cohort 3145556

October 28, 2026

Abstract

EnergyMonitor is a system for finding out how much electricity each appliance in a home actually uses. It warns when a reasonable limit is crossed and offers energy-saving tips built from the household's own readings rather than from averages borrowed elsewhere. At the physical end sits a device built around an ESP32 microcontroller and a PZEM-004T V3 sensor, which publishes telemetry every 60 seconds over MQTT 3.1.1 with TLS. On the server side, an API gateway and seven microservices written in Java 21 and Spring Boot share the workload, talk to each other through events using the transactional outbox pattern over RabbitMQ, and keep everything in MySQL. Users reach the system through a React 19 web application or an Android app built with Expo. Everything runs at https://energy-monitor.tech and is updated through continuous integration and delivery. On October 28, 2026 the automated suites were green: 1,817 backend tests, 253 web tests, 78 mobile tests, and five end-to-end journeys. Traceability covered 71 of the system's 108 requirements. Code coverage, on the other hand, had never been measured. The article does not stop at what went well: it also reports the indicators that were missed and the limitations of the work, so that readers can separate what was measured from what is still pending.

Keywords: electricity consumption monitoring, households, Internet of Things, microservices, alerts, continuous integration

Introduction

The problem

The electricity bill arrives at most homes as a single number that, in practice, says very little. You may suspect that the water heater or the air conditioner is the culprit, but you rarely know for sure, and you almost never know at what hours of the day your house concentrates its demand. Without that figure there is no way to catch obvious waste, such as a television left on in an empty room or an old unit that drinks more power than it needs. The result is a higher bill than necessary and electricity spent for no reason.

The tools that exist close the problem only halfway. Measuring at the main panel gives the whole home's aggregated consumption, not each appliance's share. Estimating from the bill hands you averages borrowed from someone else's house. And commercial smart

plugs stop at one appliance per outlet, with no alerts, no history, and no recommendations living in the same system. In all three cases what is missing is the complete loop that runs from measurement to notice, and from notice to decision.

Motivation

The core idea of the project is to measure instead of estimate. A sensor installed in the appliance itself produces readings of voltage, current, active power, and accumulated energy that reflect what that device does in that particular house; alerts and recommendations can then be built on those readings and recognized by the user as their own. Fischer (2008) studied whether feedback on household electricity consumption works as a saving tool, and that question became EnergyMonitor's design hypothesis. On the technological side, the Internet of Things supplied the scaffolding: Atzori et al. (2010) describe the integration of physical and virtual objects with identification and connectivity capabilities, while Al-Fuqaha et al. (2015) review the protocols that make that integration possible, MQTT among them.

To the product motivation we added a training one. The project belongs to the ADSO program, and we used it as a test bench to put into practice the disciplines the program teaches: requirements specification, architectural design, automated testing with traceability, continuous integration, and bilingual documentation. The article grows out of that double demand: to say what was built and what evidence backs each claim.

Objectives

The general goal was to develop an energy-resource control system capable of monitoring household electricity consumption in real time and identifying the appliances that generate the largest expense, so as to promote efficient and responsible use of electricity. That goal was broken into five specific objectives, each with a measurable indicator. Table 1 collects them together with the coverage declared in the project charter, updated on October 28, 2026.

Table 1

Specific objectives, indicators, and declared coverage

Code	Objective	Indicator and target	Coverage
OBJ-01	Measure the electricity consumption of each household appliance	One reading every 60 seconds of voltage, current, active power, and accumulated energy per connected device	Covered; device operational
OBJ-02	Present consumption clearly in a web and a mobile application	Per-home summary and history on the web; home, alert, and recommendation queries on Android in Spanish, English, French, and Portuguese	Web in production; mobile integrated with the API
OBJ-03	Issue automatic alerts on unusual or excessive consumption	Four alert types; a disconnection is detected within 150 seconds	Covered
OBJ-04	Encourage responsible habits through recommendations based on measured data	Five saving rules evaluated every hour; the same recommendation is not repeated within 7 days	Covered
OBJ-05	Deliver the system working end to end	Live demonstration, from a physical device's reading to an alert or recommendation on a phone, with no manual intervention, on October 28, 2026	System in production; demonstration scheduled for the delivery; linking a meter in production is still pending evidence

Importance of the topic

For households the importance is practical: knowing which appliances consume the most lets you react in time to excessive consumption, and savings are decided with your own data rather than with hunches, which helps control the cost of the bill. Every kilowatt-hour not wasted without need also reduces the environmental impact of electricity generation. For training, the project leaves a complete and demonstrable case: from measurement in a real appliance to the notification on the user's phone, with the documentation and metrics needed to audit it.

The topic also involves handling sensitive information: registered users, their homes, and their consumption history. That is why we built the project against Ley 1581 de 2012, Colombia's personal data protection law (Congreso de la República de Colombia, 2012). Concrete decisions follow from that reference, such as account deletion, prior email verification, and explicit acceptance of the terms at signup.

Related Work

Earlier work on domestic consumption revolves around an uncomfortable question: if the information is handed back to the user, does their behavior change? Fischer (2008) analyzed feedback on household electricity consumption as a possible saving tool, and from there came the design hypothesis of our system. The difference lies in where measurement happens. Those studies work predominantly with aggregated consumption or billing data; we instrument appliance by appliance and close the loop with alerts and recommendations inside the same system.

Something similar happened with the Internet of Things. The survey by Atzori et al. (2010) systematizes its components —identification, sensors, networks, and smart capabilities— and the applications derived from them, and the one by Al-Fuqaha et al. (2015) digs into the enabling protocols and their performance characteristics. From there came our communication decision: MQTT, in the OASIS 3.1.1 specification (Banks & Gupta, 2015), because its lightweight publish–subscribe model fits a device that publishes one reading every 60 seconds; plus TLS on port 8883 so that credentials and readings travel encrypted.

In software architecture we did not pretend to invent anything new. From Fowler (2002) we took the separation between layers and the abstraction of persistence through the Repository pattern, present in every service of the project. From Richardson (2018) came the decomposition into bounded contexts and the single entry point. The transactional outbox pattern described at microservices.io (Richardson, s. f.) solved a very concrete problem: announcing events exactly when the database write and the queue message must happen consistently. The system started life as a modular monolith and, during the final phase, its modules were extracted into independent services without changing what the user sees.

Verification rested on two international standards. ISO/IEC 25010:2023 defines the product quality model with which we organized the indicators (ISO, 2023), and ISO/IEC/IEEE 29119-3:2021 specifies the test documentation templates —plans, specifications, and reports— that our verification deliverables followed (ISO/IEC/IEEE, 2021). For API security, the checklist was the 2023 edition of the OWASP API Security Top

10 (Open Web Application Security Project, 2023), and authentication uses JSON Web Tokens signed with the RS256 algorithm, in line with RFC 7519 (Jones et al., 2015).

EnergyMonitor therefore sits at the intersection of these lines: per-appliance physical instrumentation, standard IoT transport, microservices with reliable communication patterns, and verification with requirements traceability. Its contribution is not a new algorithm. It is the complete, deployed loop, from sensor to phone, with an indicator report that covers both the targets that were met and the ones that were not.

Methodology

We ran the project as a capstone of the SENA ADSO program, with two apprentices, starting on May 1, 2026, and a delivery date that never moved: October 28. That combination —a two-person team, wide scope, immovable deadline— is exactly the scenario for which the light-methodology literature recommends direct communication and frequent deliverables over formal process (Cockburn, 2004). We shaped the plan into four phases with weekly progress and an explicit exit criterion per phase, as Table 2 summarizes.

Table 2

Project phases and exit criteria

Phase	Period	Objective and exit criterion
1. Foundation and core flow	May 1 to June 30	A registered user creates or joins a home, links a device, and sees at least one real measurement recorded.
2. Visibility and alerts	July 1 to August 31	A home that exceeds its limit produces an alert visible on web and mobile, and a notification is delivered.
3. Recommendations and internationalization	September 1 to 30	A home with one week of history receives at least one recommendation based on its own consumption, and the interface is verified in more than one language.
4. Consolidation and delivery	October 1 to 28	The system is demonstrated live, from a physical device's reading to an alert or recommendation on a phone.

The work was organized around three habits. The first was treating documentation as the source of truth: the project produced a project charter, a specification of functional and non-functional requirements with acceptance criteria, a technical proposal, the architectural and data design, a test plan, a quality report, and the user and technical manuals; everything was kept in both Spanish and English.

To documentation we added traceability. Every requirement in the specification carries a stable code —RF-SEC-13, for instance— and the test plan recorded which ones were covered by an automated test, following the test documentation logic of ISO/IEC/IEEE 29119-3:2021 (ISO/IEC/IEEE, 2021). By October 28 the specification held 108 traceable requirements.

And above all this weighed a reporting rule that governed the whole project: a number could only be presented as a result if someone had actually measured it. What had not been measured was declared as such. Code coverage and production availability therefore appear explicitly as unmeasured rather than estimated, and sections of the documentation with pending information were marked with the label [TO BE CONFIRMED] instead of being filled with assumptions. That rule explains the shape of the following sections.

Quality was worked with metrics defined from the start and with continuous integration: every repository runs its test suite in GitHub Actions on each pull request, and deployment to production requires approval of the deployment environment. Static analysis with ESLint was configured for both the web and the mobile application. The model of ISO/IEC 25010:2023 (ISO, 2023) served as a framework for evaluating product characteristics —adequate functionality, reliability, usability, performance, and security— although, as will be seen, some of those characteristics remained only partially instrumented.

Implementation

The measuring device

The device installed in the appliance combines an ESP32 microcontroller with Wi-Fi and Bluetooth (Espressif Systems, s. f.) and a PZEM-004T V3 energy meter, powered from the 120 V mains through a sealed HLK-PM03 AC-DC power supply. The ESP32 reads the PZEM-004T over a Modbus RTU serial bus at 9600 baud —reception on GPIO16, transmission on GPIO17, address 0xF8— with sensor firmware version 1.3.8. We implemented the reading on the Arduino library published for that module (mandulaj, s. f.), whose documentation reproduces the manufacturer’s specifications: 80 to 260 V, 0 to 100 A, up to 23 kW, 45 to 65 Hz, power factor from 0 to 1, energy accumulation up to 9,999.99 kWh, and a declared accuracy of 0.5%. That last figure belongs to the manufacturer: the

project never validated the sensor against a reference meter, and the limitations section returns to this point.

The firmware publishes telemetry —voltage, current, active power, and accumulated energy— every 60 seconds over MQTT 3.1.1 (Banks & Gupta, 2015) with TLS on port 8883, to the Mosquitto broker (Mosquitto, s. f.) installed on the production server and authenticated with per-device credentials. The connection status is announced every 5 minutes; if the broker receives nothing within 150 seconds, the system marks the device as disconnected and raises the corresponding alert. As protection against network outages, the firmware keeps a local buffer of 30 samples that it retransmits once connectivity returns. Linking the module to a home is done over Bluetooth Low Energy from the web application, with a configuration portal as an alternative. In money terms, the prototype closed at COP 173,997 (Table 3), below the COP 180,000 target set in the project charter.

Table 3

Components and cost of the measuring prototype (COP)

Concept	Quantity	Unit cost	Subtotal
ESP32 with Wi-Fi and Bluetooth	1	24,000	24,000
PZEM-004T-100A V3 energy meter	1	70,000	70,000
HLK-PM03 AC-DC power supply	1	20,000	20,000
Male-to-male jumper wires, 20 cm	40	300	12,000
Male-to-female jumper wires, 20 cm	40	300	12,000
Sheathed cable 2x14 AWG	3	3,333	9,999
14-gauge red and black wire	1	2,400	2,400
JST XH2.54 connector	1	1,300	1,300
Female-to-female jumper wires, 20 cm	6	383	2,298
Plastic enclosure (prototype housing)	1	20,000	20,000
Prototype total			**173,997**

The backend: an API gateway and seven microservices

The backend is deployed as an API gateway (port 8080) and seven independent microservices in Java 21 with Spring Boot 4.1.1 (Spring, s. f.): security (8082), devices (8084), measurements (8085), homes (8086), notifications (8087), alerts (8088), and recommendations (8089). Two forms of communication coexist among them. Direct queries

go through an internal API; asynchronous actions ride events on RabbitMQ (RabbitMQ, s. f.). Anyone publishing an event does it through the transactional outbox pattern: the event is written in the same database transaction as the business change, and a relay process delivers it to the queue. That way there is no window in which the data is saved without being announced, or announced without being saved (Richardson, s. f.).

Inside, every service follows hexagonal architecture: its use cases define input and output ports, and the concrete technology —persistence, email, cryptography, queue— plugs in through interchangeable adapters. The system accumulates 64 output ports for 64 contracts. The benefit shows up in testing: application tests exercise the use cases with test doubles, without starting Spring or MySQL, and swapping a technology does not touch a single line of domain code. The cost is real too, because it leaves more interfaces and more classes to maintain; in a two-person team we took that bill with our eyes open, in exchange for testability. In total, 18 design patterns were documented, with emphasis on ports and adapters, Observer, Strategy, and Repository (Fowler, 2002).

Persistence uses a shared MySQL 8.4 schema (MySQL, s. f.) with 37 tables managed by 63 changesets from the versioned migration system. Users, homes, devices, measurements, alerts, recommendations, notifications, and their support tables all live there. On top of that layer work three services worth untangling. The measurement service receives every reading, stores it exactly once —even if the device retries— and announces it. The alert service evaluates two things: consumption levels, where the 500, 1,500, and 3,000 W thresholds define the LOW, MEDIUM, HIGH, and CRITICAL states, and each home’s limits, of 6 kWh per day and 180 kWh per month. The recommendation service runs five saving rules every hour and refrains from repeating the same recommendation for seven days.

Security was resolved on several fronts, and they are worth spelling out. Authentication issues JSON Web Tokens signed with RS256 and valid for 15 minutes (Jones et al., 2015), distinguishing user sessions from administrative sessions. Authorization applies roles across 71 public routes and 14 internal API routes, and the internal routes are unreachable from the outside. The gateway limits writes to 20 POST requests per minute per IP address and route, a measure aligned with the unbounded resource-consumption risk that

OWASP documents (Open Web Application Security Project, 2023). The web server terminates TLS connections with Let's Encrypt certificates and delivers security headers: a strict content policy, X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy, and Permissions-Policy. On signup, the email is verified before the account becomes active and the terms are explicitly accepted; account deletion erases the holder's data, in line with the framework of Ley 1581 de 2012 (Congreso de la República de Colombia, 2012).

The clients

The web application is built in React 19 (React, s. f.) with Vite as the bundler and covers the full management cycle: signup and login, homes and members, device linking, consumption history and statistics, alerts, recommendations, and notification preferences. The mobile application targets Android and is built with Expo SDK 57 on React Native 0.86 (Expo, s. f.; React Native, s. f.); its scope is read-oriented —homes, device status, alerts, and recommendations— with push notifications. Both interfaces speak Spanish, English, French, and Portuguese, and verifying parity between languages is part of the mobile test suite. The web client also supports offline work: it keeps up to 30 readings in a local buffer and syncs them when connectivity returns.

Deployment and infrastructure

The system runs on a Hostinger KVM 2 VPS with Ubuntu, orchestrated with Docker Compose (Docker, s. f.): one container per service, Nginx as the web server and entry point, the Mosquitto broker, RabbitMQ, and MySQL. The domains energy-monitor.tech and mqtt.energy-monitor.tech point at the server. Every repository has a GitHub Actions workflow (GitHub, s. f.) that runs the test suite on each pull request and, from the main branch and with production-environment approval, deploys its component through an SSH script. Table 4 records the technology stack as deployed.

Table 4

Deployed technology stack

Component	Technology and version	Purpose
Device	ESP32 + PZEM-004T V3 (firmware 1.3.8), Modbus RTU 9600	Voltage, current, power, and energy readings

Component	Technology and version	Purpose
Transport	MQTT 3.1.1 with TLS on 8883; Mosquitto	Telemetry publication every 60 seconds
Backend	Java 21, Spring Boot 4.1.1: gateway + 7 microservices	REST API, business rules, and events
Messaging	RabbitMQ 4.1 with transactional outbox	Reliable events between services
Persistence	MySQL 8.4; 37 tables, 63 changesets	Single store for users, measurements, and more
Web application	React 19 with Vite	Full home and consumption management
Mobile application	Expo SDK 57 with React Native 0.86	Queries and alerts on Android
Infrastructure	Docker Compose, Nginx, GitHub Actions	VPS deployment and CI/CD with approval

Testing and Validation

That the services responded was not enough to call the system finished. Every requirement needed repeatable evidence, and that evidence had to be written down so that someone else could walk the same path. For the design of test cases and their reporting we followed the ISO/IEC/IEEE 29119-3 standard (ISO/IEC/IEEE, 2021); as a quality framework we used the product model from the SQuaRE family (ISO, 2023).

Verification strategy

The test plan divides verification into five levels. The first covers integration tests for each backend component, written in JUnit Jupiter 6.0.3 and run with Maven Surefire on every build. The second and third test the logic of the web application with Vitest 5.0.3 on jsdom, and of the mobile app with Jest inside jest-expo. The fourth assembles the full stack with Docker Compose and runs five end-to-end journeys through Bash scripts combining curl, jq, openssl, and mosquitto_pub: account registration, meter linking, publishing a measurement over MQTT, firing an alert, and account deletion. The fifth is static analysis: ESLint 9.39.4 on the web, and expo lint plus tsc --noEmit on the mobile side.

The acceptance criterion for every change was simple and strict: we did not accept any pull request with a red test suite. Table 5 summarizes the state of the five levels on October 28, 2026, the date when we ran every suite against the deployed code.

Table 5

State of automated verification on October 28, 2026

Level	Tool	Tests	Result
Backend integration (gateway and seven services)	JUnit Jupiter 6.0.3, Maven Surefire	1,817 in total: 774 security, 260 homes, 235 notifications, 151 measurements, 129 devices, 128 alerts, 71 recommendations, and 69 gateway	Green
Web unit tests	Vitest 5.0.3 (jsdom)	253 tests in 44 files	Green
Mobile unit tests	Jest (jest-expo)	78 tests in 8 suites	Green
End-to-end integration	Bash, curl, jq, openssl, and mosquitto_pub on a local Docker Compose stack	160 assertions across 5 journeys: device (24), measurement (28), alert (36), activity (60), and account deletion (12)	Green; the recommendation journey was not re-run in version 1.3
Static analysis and types	ESLint 9.39.4 (web); expo lint and tsc --noEmit (mobile)	Not applicable	0 findings

The distribution of those 1,817 backend tests is no accident. Security holds 774, about 43% of the total, because that is where email-verified registration, terms acceptance, single-use refresh tokens, role-based access control, and user administration live: precisely the front that OWASP (2023) ranks as the most attacked in application programming interfaces. Homes and notifications, with 260 and 235 tests, reflect that membership rules and delivery preferences are among the messiest parts of the domain.

The static analysis result deserves a clarification. Zero ESLint findings across two applications speaks about team discipline, not about the absence of defects: ESLint catches forbidden patterns and typed errors, not wrong logic. The metric that would have closed this chapter was code coverage, and we do not have it. JaCoCo was never configured; it was logged as high-priority technical debt (DT-11) and is acknowledged here as a limitation.

Requirements traceability

The specification document handles 108 stable requirements, with codes identical in the Spanish and English versions. Of these, 71 are exercised by an automated test —65.7%. The remaining 37 cluster around four fronts the project knows well: the mobile application, with 78 unit tests and still no component or screen tests; the ESP32 firmware; Bluetooth integration; and push notification delivery. The manual protocol for the mobile application,

twelve tests numbered MOV-01 to MOV-12, is also documented but unexecuted. The gap is described, not hidden, and the limitations section returns to it.

Validation in the production environment

Besides the local suites, we measured the already-deployed system on October 28, 2026, between 12:33 and 12:34 UTC. The landing page answered HTTP 200 in 10 of 10 requests, with a time to first byte averaging 0.36 s once the web server was warm and 0.41 s including the first cold start. The gateway route /api/v1/consumption-levels answered HTTP 401 on all five test requests, averaging around 0.37 s. That rejection is exactly the expected behavior, because the route requires a JSON Web Token, and it serves as evidence that the API does not hand data to unauthenticated callers (Jones et al., 2015; Open Web Application Security Project, 2023).

The same round checked the edge defenses. Nginx 1.28.3 on Ubuntu returned the content policy header, X-Content-Type-Options, X-Frame-Options in DENY mode, Referrer-Policy, and Permissions-Policy. The host mqtt.energy-monitor.tech accepted the TCP connection and completed the TLS handshake on port 8883. ICMP did fail, but the provider's firewall discards it systematically, so that failure does not indicate unavailability. It should be said plainly: these measurements are point-in-time. They do not replace the continuous monitoring of latency and availability that the project still lacks (DT-20), nor do they validate requirement RNF-REN-01, which demands under 2 seconds for authenticated queries with data; measuring it would have required a valid session and representative load.

Results

We grouped the results into two blocks: how the quantitative targets set in the project charter and specification were met, and the actual state of software quality at the close. Table 6 gathers the indicators with their documented target, the measured value, and the status. Where a target does not exist or is marked as pending confirmation, the table says so instead of inventing a number.

Table 6

Summary of quantitative results, status on October 28, 2026

Indicator	Documented target	Measured value	Status
Hardware prototype cost	No more than COP 180,000 (charter of September 23, 2026)	COP 173,997 for items 3 to 6 of the shopping list	**Met** (96.7% of the budget)
Telemetry cadence	One reading every 60 seconds (RNF-REN-02)	Firmware parameter set to 60 s	Implemented; live demonstration on October 23
Disconnection detection	150-second threshold (RNF-DIS-02)	150 s window implemented on the device	Implemented
Server outage tolerance	Local queue of 30 readings (RNF-DIS-03)	Circular buffer of 30 samples on the ESP32	Implemented
API authentication	Rejection without credentials (RNF-SEG-01)	5 of 5 requests answered HTTP 401 (10/28/2026)	**Met**
Query latency	Under 2 s with authenticated data (RNF-REN-01)	0.37 s on unauthenticated rejections; no measurement with data	Partial; measurement pending
Landing page latency	No quantitative target	0.36 s to first byte (10 requests, 10/28/2026)	Measured at a point in time
Service availability	No target (RNF-DIS-01 marked [TO BE CONFIRMED])	No continuous measurement (DT-20)	Pending
Backend tests	Green suites before every merge	1,817 tests green (10/28/2026)	**Met**
Web and mobile tests	No numeric target; existence of automated suites	253 (web) and 78 (mobile) green	**Met**; no component tests
End-to-end tests	Five automated journeys on the local stack	160 assertions green	Partial; one journey not re-run
Static analysis	No ESLint findings	0 findings on web and mobile	**Met**; not yet in CI
Requirement–test traceability	No documented numeric target	71 of 108 requirements (65.7%)	37 requirements uncovered
Code coverage	Threshold to be defined (MP-02)	Not measured: JaCoCo not installed (DT-11)	Not met

On the money side, the prototype closed at COP 173,997, with COP 6,003 of margin under the charter's ceiling. The full system shopping list comes to COP 8,073,997 in hardware and COP 48,000 in software, for a total of COP 57,021,997 covering every meter, controller, electronic supply, and contracted service. Operation runs on a single Hostinger KVM 2 server with four cores and 8 GB of memory, with no load balancers or replicas: the

same decision that made it possible to publish version 1.0 in August and that is now documented as an accepted risk.

What convinces us most is not a single figure but continuity. Between October 24 and 28, 2026 we spread the monolith —Spring Modulith— across a gateway and seven services with no downtime windows, keeping the same domain names and pointing the web and mobile apps at the same address as always. The fourth-level suites, run against the full stack, stayed green during and after the migration; the version 1.3 deployment on October 28 closed with 160 end-to-end assertions green.

Discussion

The data allow one concrete claim: the product's full cycle works. A COP 173,997 meter takes readings every minute, carries them encrypted over MQTT, stores them in MySQL, crosses them against five hourly rules, publishes recommendations, and raises disconnection alerts within the 150-second threshold. That journey was demonstrated live before the evaluation committee on October 23 and is reproducible with the end-to-end scripts. What the data do not allow is claiming that the system saves its users energy: there is no study with real users, nor before-and-after consumption measurements, nor perception surveys. The domestic feedback literature (Fischer, 2008) warns that measuring by itself does not guarantee savings. This project delivers the measurement; studying its effect remains future work.

Hexagonal architecture had a practical consequence we did not budget for: testability. The 1,817 backend tests run on a local machine without depending on cloud services, because each service exposes its use cases through ports that tests substitute at will (Fowler, 2002; Richardson, 2018). When the microservices split arrived, that saving came due: breaking eight components apart in two weeks without outages would have been unthinkable if every service had been born tied to its database or to remote-environment credentials. The same design underpins the transactional outbox pattern on RabbitMQ, which guarantees that a stored measurement ends up published without duplicates even if the bus goes down halfway (Richardson, s. f.).

Technical debt is the other thread of the discussion. The project documents 25 findings with their severity and fix cost, and four remain open at high priority: non-hermetic tests (DT-07), unmeasured coverage (DT-11), backups living on the same server as production (DT-19), and the lack of latency and availability monitoring (DT-20). None of the four prevents using the system; all four limit it in an adverse scenario. Owning them out loud, with codes and dates, is part of the same honesty that leads this article to state that availability is not measured and that sensor accuracy was never validated against a reference pattern.

Against the reviewed work, the contribution lies in the cost of the closed loop. Internet of Things surveys describe the field as a whole (Atzori et al., 2010; Al-Fuqaha et al., 2015), and home energy monitoring appears there as a consolidated use case; this article shows its low-cost, open-source variant, built on an ESP32 microcontroller (Espressif Systems, s. f.) and a PZEM-004T V3 sensor whose Modbus interface is supported by a community library (mandulaj, s. f.). Choosing MQTT 3.1.1 over TLS (Banks & Gupta, 2015; Mosquitto, s. f.), with device authentication via hash and short-lived RS256-signed tokens (Jones et al., 2015), answers directly the credential-exposure and unauthenticated-interface risks that OWASP (2023) documents for APIs.

The last point is the team. Two apprentices sustained twelve repositories, a 37-table database, eight services, and two client applications for five months. Crystal Clear (Cockburn, 2004) was chosen for exactly that team size, and its practices of frequent delivery and peer review translated into dev and main branches, pull requests with approval, and a continuous integration and delivery channel per repository (GitHub, s. f.). The cost of that ambition shows in the 23-minute CI run and in manual SSH deployment from the server's root (DT-21 and DT-25); the benefit, in that every deployed component at the close has its own pipeline and its own version history.

Conclusions

Summary of findings

1. EnergyMonitor reaches the general goal it set: measuring each appliance's electricity consumption, presenting it on a web and a mobile application, issuing alerts, and

recommending saving actions. The prototype closed at COP 173,997, COP 6,003 under the approved budget.

2. The system has been deployed and running at energy-monitor.tech since August 2026. On October 28 it moved to running as a gateway and seven microservices without switching the service off for a single minute.
3. The figure that sums up the verification work is 2,148 automated tests —1,817 backend, 253 web, 78 mobile— plus 160 assertions across five end-to-end journeys. All were green at the close, along with a static analysis run with no findings.
4. Not everything is covered. Traceability reaches 71 of the 108 requirements; the remaining 37 concentrate in the mobile application, firmware, Bluetooth, and push notifications, and they are documented.
5. In production, point-in-time measurements confirm that the API rejects unauthenticated requests, that the web server's security headers are active, and that the landing page answers in about 0.36 s. What is not continuously measured, on the other hand, is availability and latency under load.
6. And the technical debt was not hidden: 25 findings were logged with severity, cost, and a fix plan. The four high-priority ones remain open and declared.

Limitations

The PZEM-004T V3 sensor was never validated against a reference meter, and that is the biggest limitation of the work. The 0.5% accuracy invoked here is the manufacturer's, not a laboratory measurement: the system reports with catalog precision, not instrument precision. Software quality, for its part, is measured only where instruments exist. Test counts and ESLint findings exist, but there is no code coverage (DT-11), no SonarQube analysis, and no performance or accessibility testing; the latter were marked as pending in the specification itself.

Operationally, the system runs on a single server with no staging environment, backups sit on the same machine that serves the product (DT-19), and availability is not monitored (DT-20). Local tests are not hermetic either: Maven profiles download dependencies from external repositories and the end-to-end flows need the full container

stack raised by hand (DT-07). Telemetry transport uses QoS 0, a level with no delivery guarantee — a decision that works so far but would be insufficient if the home network turned hostile. And the device has no over-the-air updates: fixing its firmware requires physical intervention.

The latest features did not reach both applications equally either. The web already has the Settings help center (documentation by role, support for users and news), member photos with their viewer and the notice to enable push in Brave; mobile does not yet open those modals, shows only initials for members and renders the unusual consumption alert with the generic threshold text. In addition, the thresholds of that alert are fixed in the code (DT-27) and the news titles come out in English (DT-26).

Finally, real-usage evidence is missing. There are no production users, no surveys, and no before-and-after consumption studies after adopting the system, so no energy-saving claim can hold. The mobile application's manual protocol (MOV-01 to MOV-12) went unexecuted, React Native component tests do not exist yet, and the end-to-end recommendation flow was not re-run after the services split. The integration and delivery pipeline was measured at 23 minutes against the monolith's 1,131 tests; after separating the services it was never timed again.

Future Work

Software quality

The first task is closing the four high-priority debts. Configuring JaCoCo across the eight backend components and setting a minimum coverage threshold closes DT-11 and turns the test count into a comparable indicator (MP-02); moving integration tests to Testcontainers removes the dependence on local profiles and makes the suites hermetic (DT-07). On the mobile side we want to add React Native Testing Library to test components and screens, and to run the MOV-01 to MOV-12 protocol on a physical device. On the web, the priority is an accessibility analysis against WCAG, currently without evidence.

Operations

Backups must move off the production server to external storage, with a documented restore test (DT-19), and the system needs latency and availability monitoring with alerts to

the team itself (DT-20). That would also allow requirement RNF-REN-01 to be met with authenticated data and representative load, something that cannot be claimed today. The web ESLint should move into the pipeline as a blocking step, deployment should stop using the server's root account (DT-25), and a testing stage between dev and main is worth exploring.

Product and research

On the product side, the roadmap already includes over-the-air firmware updates, a smart meter for users without Arduino, cost estimates in local currency based on the current tariff, and an iOS application, out of scope until now. On the research side, the most promising line is measuring real impact: a study with volunteer households logging consumption before and after adopting the system, with perception surveys, and cross-validating the PZEM-004T against a reference meter to turn catalog accuracy into verified accuracy.

Appendix A

Reproducibility

The system is reproducible from the project's public repositories. Every component has a main production branch and a dev integration branch; pull requests go first to dev and from dev to main. Table A1 lists the active repositories and the main commit as of October 28, 2026, the day of the complete migration to microservices; those marked "deployment to be confirmed" are already in main but their deployment on the server had not been confirmed at the close. The retired repositories, energy-monitor-backend (which keeps the ESP32 firmware in its firmware/ folder) and energy-monitor-devices, are kept as history.

Table A1

Active repositories and main commit as of October 28, 2026

Component and repository	Commit	Production status
API gateway (en-monitor-api-gateway)	`0b2c164`	Deployed
Security service (en-monitor-security-service)	`de52c95`	In main; deployment to be confirmed
Device service (en-monitor-device-service)	`06c1bdc`	Deployed
Measurement service (en-monitor-measurement-service)	`8a9b544`	Deployed

Component and repository	Commit	Production status
Home service (en-monitor-home-service)	`6a4e396`	In main; deployment to be confirmed
Notification service (en-monitor-notification-service)	`ecdbc0a`	In main; deployment to be confirmed
Alert service (en-monitor-alert-service)	`0e65430`	In main; deployment to be confirmed
Recommendation service (en-monitor-recommendation-service)	`aeeeec5`	Deployed
Web application (Energy-Monitor-application)	`27ad9f0`	In main; deployment to be confirmed
Infrastructure (energy-monitor-infra)	`ceb1543`	Deployed
Mobile application (Energy-Monitor-Mobile)	`894b65a` on main; `8b5edc0` on dev	Not applicable; not deployed on the server

To repeat the verification, each backend repository runs its tests with mvn test inside its own pipeline job; the web runs npm test with Vitest and the mobile npx jest. The end-to-end flows live in the e2e/ directory of energy-monitor-infra and are launched with Bash against a stack raised with Docker Compose. Repeating the production measurements requires just two command-line tools: curl with timing options for the landing page and the API, and openssl s_client against mqtt.energy-monitor.tech:8883 to check the TLS handshake. The values reported here correspond to the public environment at energy-monitor.tech on the date indicated above.

References

Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, *17*(4), 2347–2376. https://doi.org/10.1109/COMST.2015.2444095

Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, *54*(15), 2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010

Banks, A., & Gupta, R. (Eds.). (2015). *MQTT version 3.1.1*. OASIS Standard. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

Cockburn, A. (2004). *Crystal clear: A human-powered methodology for small teams*. Addison-Wesley.

Congreso de la República de Colombia. (2012). *Ley 1581 de 2012 (17 de octubre): Por la cual se dictan disposiciones generales para la protección de datos personales* [Organic Statute 1581 of 2012 on general provisions for the protection of personal data]. Diario Oficial 48587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Docker. (s. f.). *Docker Compose*. Recuperado el 23 de septiembre de 2026, de https://docs.docker.com/compose/

Espressif Systems. (s. f.). *ESP32: Wi-Fi & Bluetooth MCU*. Recuperado el 23 de septiembre de 2026, de https://www.espressif.com/en/products/socs/esp32

Expo. (s. f.). *Expo SDK 57*. Recuperado el 23 de septiembre de 2026, de https://docs.expo.dev/versions/v57.0.0/

Fischer, C. (2008). Feedback on household electricity consumption: A tool for saving energy? *Energy Efficiency*, *1*(1), 79–104. https://doi.org/10.1007/s12053-007-9002-5

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

GitHub. (s. f.). *GitHub Actions: Build, test, and deploy your code right from GitHub*. Recuperado el 28 de octubre de 2026, de https://docs.github.com/en/actions

ISO. (2023). *ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*.

International Organization for Standardization. https://www.iso.org/standard/35733.html

ISO/IEC/IEEE. (2021). *ISO/IEC/IEEE 29119-3:2021: Systems and software engineering — Software testing — Part 3: Test documentation*. International Organization for Standardization. https://www.iso.org/standard/45904.html

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT): RFC 7519*. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7519

mandulaj. (s. f.). *PZEM-004T-3.0 Arduino library*. GitHub. Recuperado el 23 de septiembre de 2026, de https://github.com/mandulaj/PZEM-004T-3.0

Mosquitto. (s. f.). *The Mosquitto MQTT broker*. Recuperado el 23 de septiembre de 2026, de https://mosquitto.org/

MySQL. (s. f.). *MySQL 8.4 Reference Manual*. Recuperado el 23 de septiembre de 2026, de https://dev.mysql.com/doc/refman/8.4/en/

Open Web Application Security Project. (2023). *OWASP API Security Top 10 (2023)*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

RabbitMQ. (s. f.). *RabbitMQ docs*. Recuperado el 23 de septiembre de 2026, de https://www.rabbitmq.com/docs

React. (s. f.). *React – The library for web and native user interfaces*. Recuperado el 23 de septiembre de 2026, de https://react.dev/

React Native. (s. f.). *React Native 0.86 documentation*. Recuperado el 23 de septiembre de 2026, de https://reactnative.dev/

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Richardson, C. (s. f.). *Transactional outbox*. microservices.io. Recuperado el 23 de septiembre de 2026, de https://microservices.io/patterns/data/transactional-outbox.html

Spring. (s. f.). *Spring Boot 4.1.1 reference documentation*. Recuperado el 23 de septiembre de 2026, de https://docs.spring.io/spring-boot/reference/